EXACT CODE. NATURAL LANGUAGE. ONE FILE.

Less plumbing. More purpose.

A Python-like language with LLM calls built in. Forget the callback hell of agentic frameworks.

Executable Markdown | Shared variables | Explicit guards

A WORKFLOW YOU CAN READ, NOT A GRAPH TO UNTANGLE

```
let notes = read_file('notes.txt')
let summary = ask(len='sentence') 'Summarize: {notes}'
```

Build reliable automations.
Keep control flow deterministic. Validate model output. Record decisions. Bound actions.

Python-like syntax, not Python itself. The interpreter runs on Node.js; model calls use GitHub Copilot.

Your first working workflow.

Node.js 24.15.0+ for the CLI and notebook runtime. GitHub Copilot credentials for model calls.

1

Install from npm

Install the CLI and authenticate:

```
npm install -g spl-lang
spl login
```

Open this repository checkout in VS Code. The npm package installs the runtime, not this repository's example files.

2

Set up VS Code

Install SPL - Semantic Prompt Language by **spl-lang** from Extensions.

```
code --install-extension spl-lang.spl-lang-vscode
```

Requires VS Code 1.82+. Get syntax highlighting, diagnostics, completions and runnable Markdown notebooks.

3

Run the basics

From the repository root:

```
cd lang
spl examples\basics.md --check
spl examples\basics.md
```

Or open `lang\examples\basics.md`, run **SPL: Open as Notebook**, select **SPL Runtime**, then run cells in order in a trusted workspace.

What you'll see: exact functions, agent instructions, shared values and an assertion comparing Fibonacci results. `--check` validates syntax without calling a model; live calls use the example's configured model.

Sources: Getting started / VS Code / basics.md

Two executable surfaces. Zero magic glue.

Only `spl` fences and blockquotes execute. All other Markdown is commentary.

double.md / a complete, minimal workflow

````
---
extension: semantic-prompt-language
version: 1
---
```spl
def double(n):
    return n * 2
```

> Invoke @double with 21 -> @answer

```spl
assert answer == 42 'wrong result'
success answer
```
````

WHAT THE BLOCKQUOTE DOES INTERNALLY

It is an agent command. Not generated SPL.

The model chooses tools against the real interpreter. For this instruction, the intended tool sequence is:

1. **`call_function('double', [21])`**
   Invokes the existing deterministic function.
2. **`save_variable('answer', 42)`**
   Saves the result into the shared scope.
3. The next fence checks **`answer`**.
   The model's prose alone never creates a variable.

> **Exact fences:** Python-like `:` + indentation, `let` bindings and deterministic control flow. **ask / think** generate values without tools; **agent / blockquotes** can act.

`@name` marks a binding or operation, not text substitution. `let` is local to a block; assignment updates the nearest declaration.

Sources: Natural language & models / Language reference

Keep decisions. Recover deliberately.

Saved model responses reduce repeat work. Checkpoint journals and trace replay solve different problems.

CHECKPOINT / CONTINUE THE SAME RUN

Preserve completed work

```
spl flow.md --checkpoint run.jsonl
spl flow.md --resume run.jsonl
spl flow.md --resume run.jsonl --answer-for REQUEST_ID --answer "Approved"
```

A human question exits **3** with JSON; answer using its `request_id`. Resume reconstructs completed steps without repeating completed tool/model calls.

> After a crash, resume the same journal in the same workspace with the same script, runtime, options and environment. An interrupted action requires manual recovery; the journal is not a filesystem snapshot.

TRACE / RE-TEST AGAINST SAVED ANSWERS

Replay code and guards

```
spl flow.md --trace calls.jsonl
spl flow.md --replay calls.jsonl
```

Replay returns recorded responses without contacting Copilot or MCP. Requests must match exactly, in order; changed prompts require a new trace.

> **Not a dry run:** explicit file, shell and web tools still execute. Agent-owned external effects are not repeated or recreated. Keep traces and journals private; they contain prompts and answers.

Human input, two ways

Normal runs prompt in the terminal. Repeat `--human` to answer in question order; remaining questions still prompt. Blockquotes use `ask_human` via `@HUMAN`.

```
let feedback = human 'What should change?'
# CLI: prompt, or supply an answer
spl review.md
spl review.md --human "Make it clearer"
```

Choose a mode: `--human` cannot combine with checkpoint/resume; checkpoints cannot combine with trace/replay or variable stores.

Sources: CLI & runtime / Human in the loop

Guide the model. Restrict the actions.

MCP extends agent tools. Runtime restrictions define which actions those agents may take.

SOFT INSTRUCTIONS / HARD CHECKS

SOFT **`enforce` / `forbid`** add prompt guidance to nested model calls. They do not validate an answer or guarantee compliance.

HARD **`restrict`** checks runtime actions. Blocked calls raise `restriction_violation`. Use schemas and `assert` to validate data.

```
restrict fs './out', mcp 'docs/search':
    let report = agent 'Use only allowed tools'
```

Restrictions can also limit network hosts and executables. File paths, redirect hosts, executable names and MCP server/tool names are checked. Nested scopes intersect: they can only narrow access.

> **Know the boundary:** unrestricted categories are allowed by default. Restrictions are not an OS sandbox; allowed programs and MCP servers remain trusted.

MCP / WIRE IT ONCE, INVOKE THROUGH AN AGENT

Expose a docs search tool

Example `mcp.json` (VS Code format):

```
{"servers":{"docs":{
  "type":"stdio", "command":"node",
  "args":["tools/docs-server.js"],
  "tools":["search"]
}}}
```

Provide your own trusted server script exposing `search`; this config is illustrative. Keep credentials in environment variables, never in the config.

```
spl flow.md --mcp mcp.json

let answer = ''
restrict mcp 'docs/search':
    answer = agent 'Use docs/search to find setup docs'
```

Configured MCP tools are available to **agent expressions and blockquotes only**, never `ask` or `think`. Out-of-scope MCP tools are hidden and rejected if requested.

Sources: Restrictions / MCP servers / mcp.md